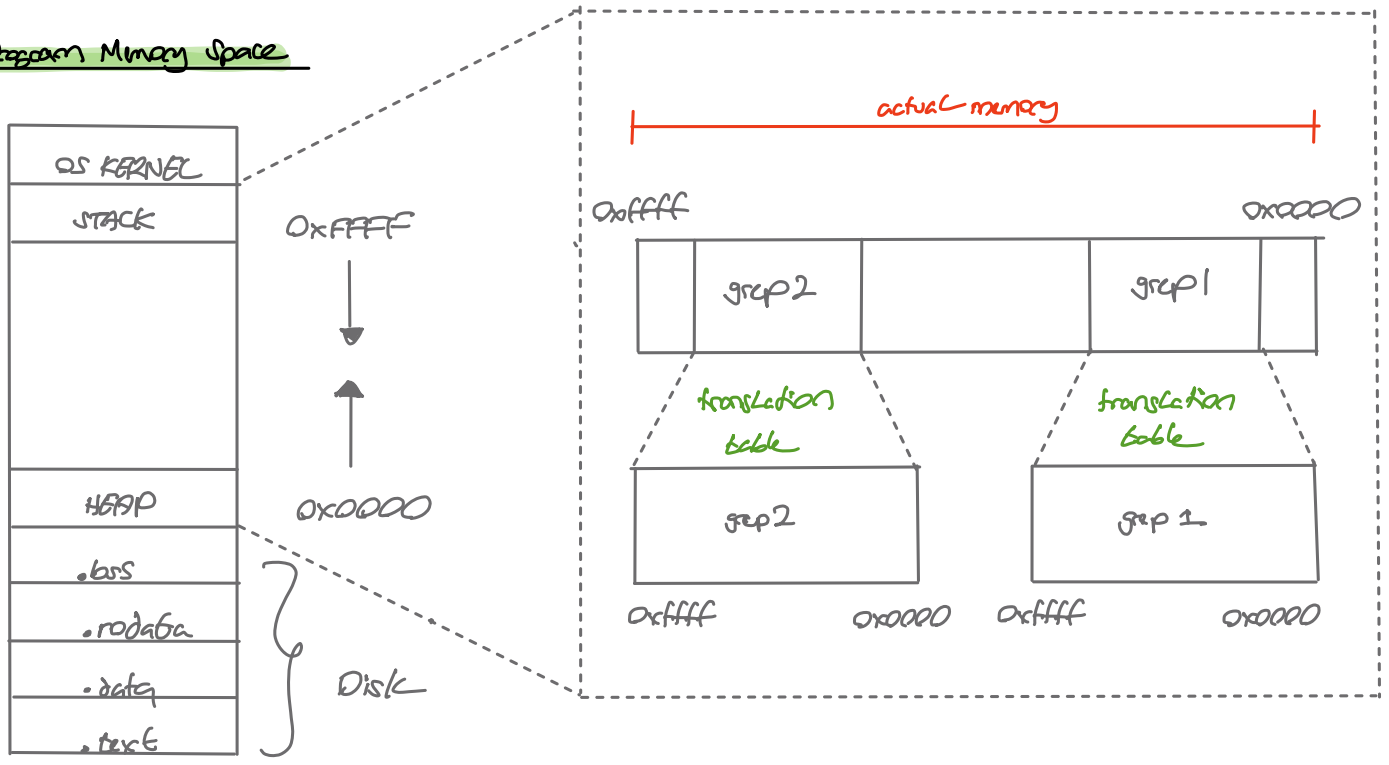
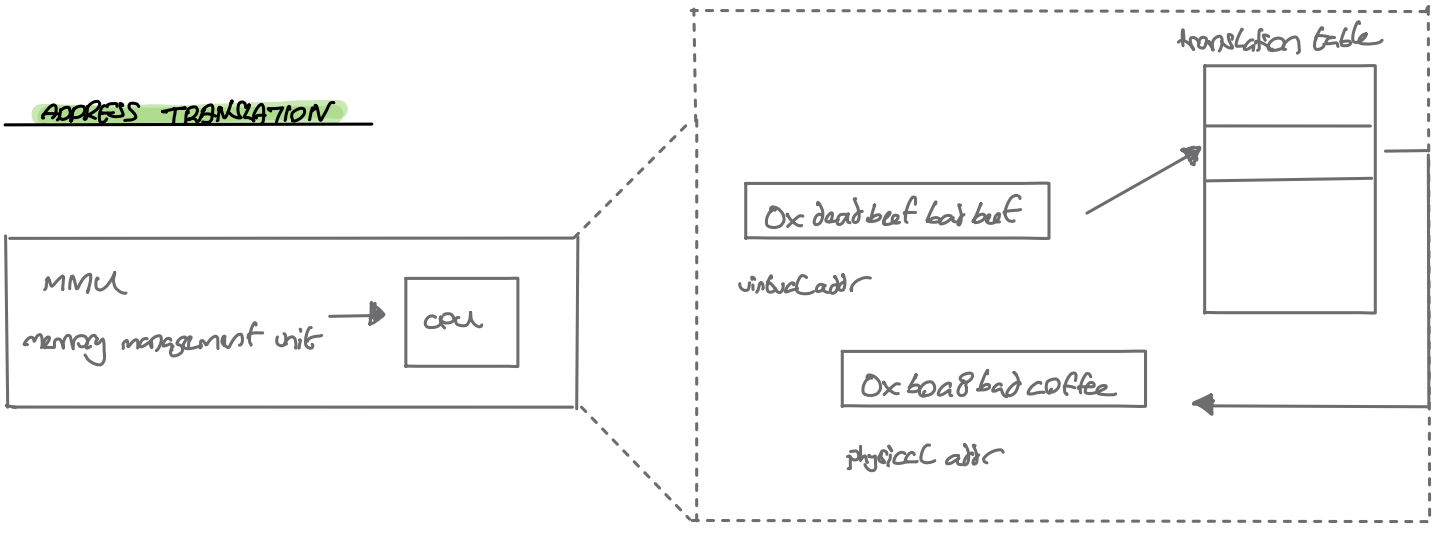
# Virtual Memory

## Virtual Memory Abstraction




1. Each program has a seperate VMem space
2. All memory accesses are virtual memory addresses
3. This is mapped to physical mem. addresses.

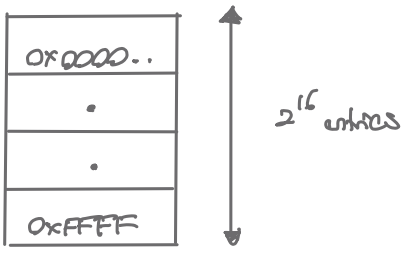
## ADDRESS TRANSLATION




a). page size: 4KB ~ $2^2 \cdot 2^{10} = 2^{12}$

b). virtual page number: 36 bits

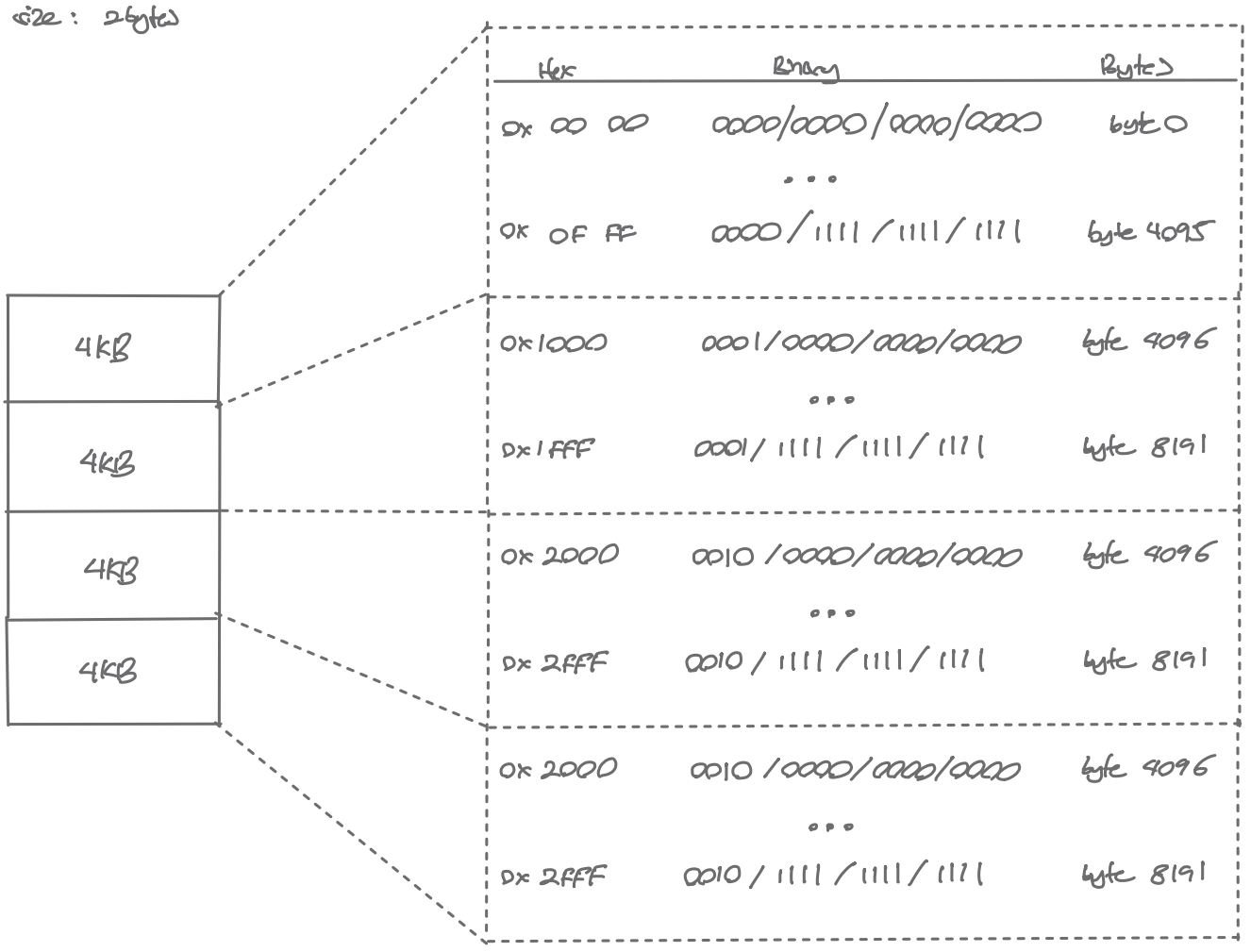





size: 2 bytes

**Key Concept:**

Instead of managing memory byte by byte; OS cuts it down into equal sized blocks = pages




**Key Concept:**

- Each page holds 4096 bytes
- 12 bit offset → "which byte inside the current page?"

- Memory has no intrinsic meaning.
- RAM stores bits ~ 0000 1111 ...
- Meaning comes from interpretation. ⟶ bytes can represent text, numbers, instructions, ptrs, floating point.

Example: Integers

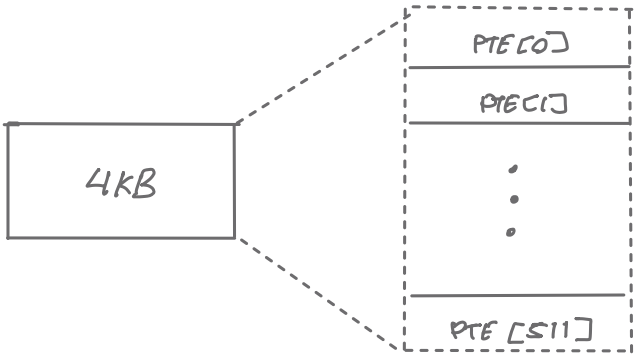
- A page ~ (4096 bytes) can hold integers instead.

| Address | Byte | |
|---|---|---|
| 0x1000 | 0x48 | 'H' |
| 0x1001 | 0x65 | 'E' |
| 0x1002 | 0x6C | 'L' |
| 0x1003 | 0x6C | 'L' |
| 0x1004 | 0x6F | 'O' |

Example: Page Table

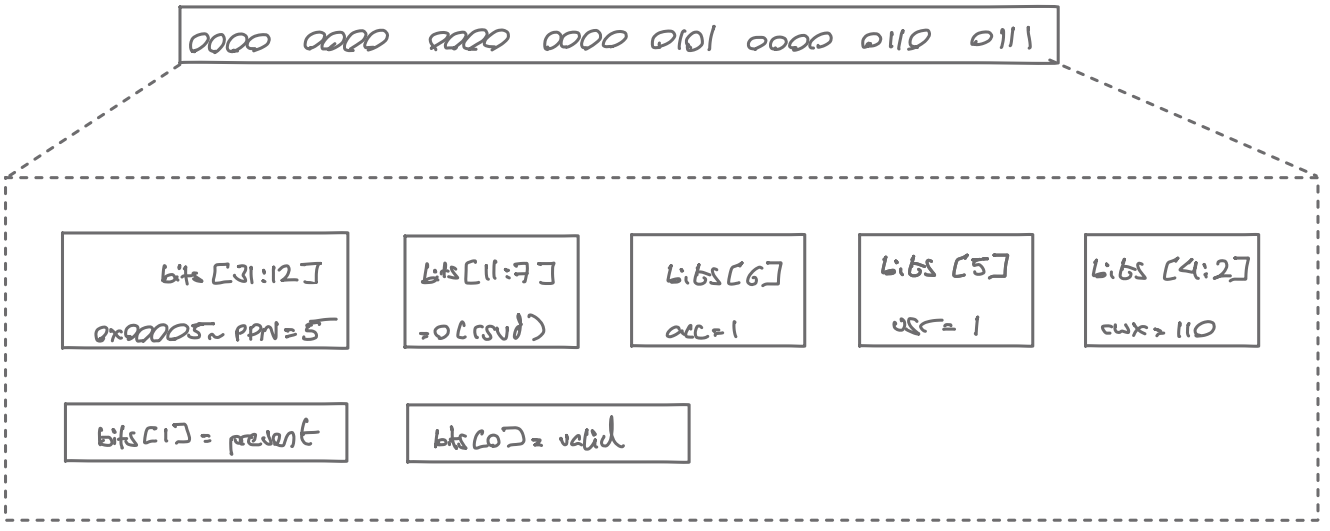
- A page can also be used as a page table
- If each PTE is 8 bytes

$4KB = 2^{12} \cdot \frac{4096\,B}{8\,B} = 512$ entries.




Page Table Entry

example: pte[0] contains <0x00 00 50 67> (32 bit virtual address)




Key concepts

- virtual page 2 maps to physical page 5, it is valid and present in RAM.
- it is readable and writable but not executable. user accessible. has been accessed.

## One Level (VPN)

a. <0x00000ABC> (VPN=0)

b. lookup VPN[0] = PTE[0] → (PPN=5)

c. physical addr = (PPN · page size) + offset

physical addr = 5×4096 + 0xABC = (0x5ABC)

## Multi Level (VPN)

example: <0x00402ABC> → { offset: 12 bits → $2^{12}$ = 4096 bytes; VPN = 20 bits

one level page table: $2^{20}$ : 1,048,576 entries

each PTE = 4 bytes : $2^{20} \cdot 4B = 2^{22}$ = (4 MiB)

key concepts: - every concept would need a 4 MiB page table:

even if it uses a small amount of memory.

two level page table:

4KB pages = $2^{12}$, 4096 } $\frac{2^{12}}{2^2} = 2^{10}$ → each table = $2^{10}$, 2 levels of ($2^{20}$).

each PTE = 4 bytes

virtual addr: <0x00402ABC>

| bits [31:22] = 1<br>L1 index | bits [21:12] = 2<br>L2 index | bits [11:0] = 0xABC<br>offset |
|---|---|---|

L1 table @ base 0x1

| slot 0: valid=0 (empty)<br>slot 1: PPN: 7 → L2 table<br>slot 2: valid=0 (empty) |
|---|

L2 table @ (base + L2 idx · pte size)

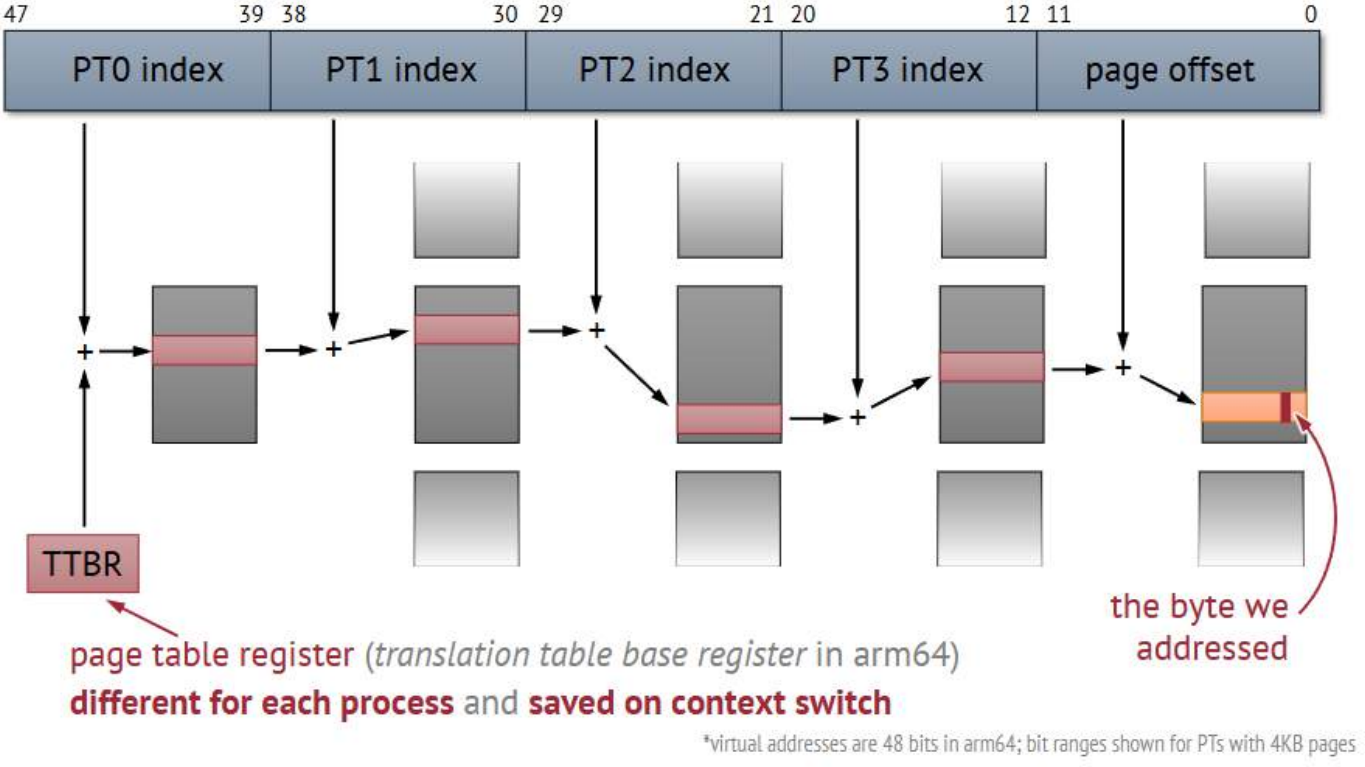
| slot 0: valid=0 (empty)<br>slot 1: valid=0 (empty)<br>slot 2: PPN: 5 → data |
|---|

data page @ (PPN · page size + offset)

| + offset 0xABC<br>= 0x5ABC |
|---|

* virtual addresses are 48 bits in ARM 64

0x ABCD EF12 3456




## Memory usage

flat table: 48 bits ~ offset ~ $2^{12}$ → 36 bits ~ $2^{36}$ → 512GB per process

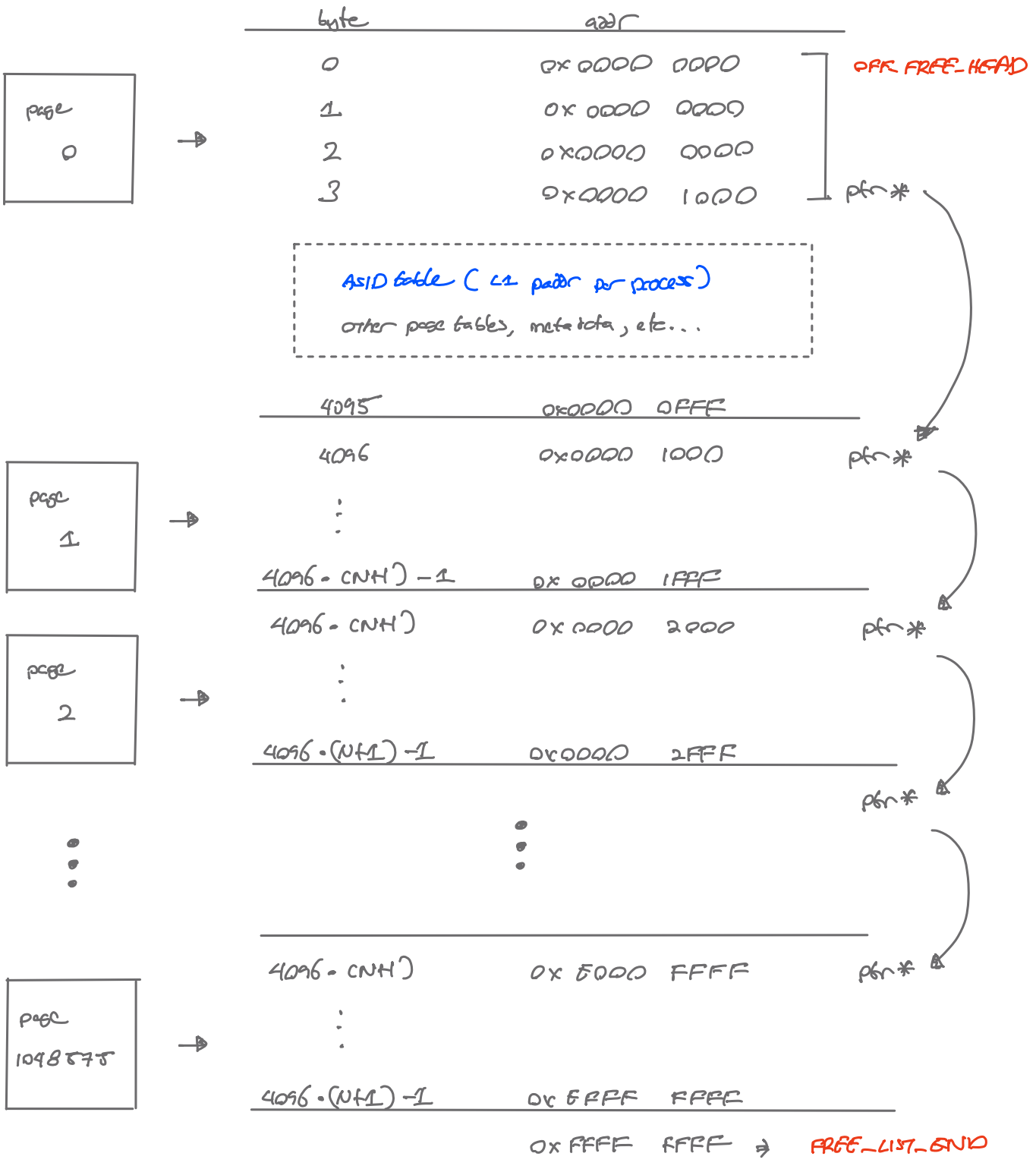
4 level ~ L0: 29 → 512 · 8 bytes → 4KB
L1: 29 → 512 · 8 bytes → 4KB
L2: 29 → 512 · 8 bytes → 4KB
L3: 29 → 512 · 8 bytes → 4KB

16KB per process (worst case)
< 16KB; you only allocate tables you need. **

* 4 level table uses 33 million times less memory than FLAT TABLE.

## Physical Memory Layout

- 32 bit addresses = [0x 0000 0000 – 0x FFFF FFFF]
- page size = 4KB = $2^{12}$ B + $(2^4)^3$ + $(16^3)$ → 0x1000
- num_physical_pages = $\frac{2^{32}}{2^{12}}$ = $2^{20}$ = 1,048,576 pages



# 32 bit - VIRTUAL ADDRESS

